Visualisatie van de control flow in HLASM-code met Mermaid.js.

Ter ondersteuning van beginnende ontwikkelaars.

Sarah Bader.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. L. Blondeel
Co-promotor: Dhr. B. Tjassens Keiser
Academiejaar: 2025–2026
Eerste examenperiode

Departement IT en Digitale Innovatie .

HO
GENT

Woord vooraf

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Inhoudsopgave

Lijst van figuren viii

Lijst van tabellen ix

Lijst van codefragmenten x

1 Inleiding 1
- 1.1 Probleemstelling . . . 1
- 1.2 De onderzoeksvraag . . . 2
- 1.3 Onderzoeksdoelstelling . . . 2
- 1.4 Het verwachte resultaat . . . 3
- 1.5 Structuur van het onderzoek . . . 3

2 Literatuurstudie 4
- 2.1 De mainframeindustrie . . . 4
  - 2.1.1 Modernisatie van legacy-systemen . . . 5
  - 2.1.2 Eigen Bijdrage . . . 5
- 2.2 Aanwezigheid van HLASM-code . . . 5
  - 2.2.1 Onderhoud en Modernisatie . . . 5
  - 2.2.2 Bestaande HLASM-hulpmiddelen . . . 6
  - 2.2.3 Kosten van Onderhoud . . . 7
- 2.3 Moeilijkheden voor Beginnende Ontwikkelaars . . . 7
  - 2.3.1 Eigen Bijdrage . . . 8
- 2.4 Grafische Softwarevisualisatie . . . 8
  - 2.4.1 Control-Flow Grafieken . . . 8
  - 2.4.2 Mermaid.js . . . 8
  - 2.4.3 Eigen Bijdrage . . . 8
- 2.5 Parsing van programmeertalen . . . 9
  - 2.5.1 JavaScript Parsing . . . 9

3 Methodologie 10
- 3.1 Fase 1: Literatuurstudie . . . 10
- 3.2 Fase 2: Technische analyse van HLASM . . . 10
- 3.3 Fase 3: Requirementsanalyse . . . 11
- 3.4 Fase 4: Ontwerp en Implementatie . . . 11
- 3.5 Fase 5: Evaluatie . . . 11

4 HLASM-analyse **13**
4.1 Structuur van een HLASM-programma . . . 13
4.1.1 Een Assembler Statement . . . 13
4.1.2 Instructies . . . 13
4.1.3 Assembler Directives . . . 16
4.2 Control-flow in HLASM . . . 16
4.3 Selectie van control-flowrelevante instructies . . . 17

5 Control-flow visualisatie **18**
5.1 Requirements . . . 18
5.1.1 Functionele vereisten . . . 18
5.1.2 Niet-functionele vereisten . . . 19
5.2 Ontwerp . . . 19
5.2.1 Mapping van HLASM naar Mermaid.js . . . 19

6 Proof-of-Concept **20**
6.1 Architectuur van de applicatie . . . 20
6.2 Beperkingen van de implementatie . . . 21
6.3 Evaluatie . . . 22
6.3.1 Titel A . . . 23
6.3.2 Titel B . . . 24
6.3.3 Titel C . . . 26

7 Conclusie **28**

A Onderzoeksvoorstel **30**
A.1 Inleiding . . . 30
A.2 Literatuurstudie . . . 31
A.2.1 Belang van de mainframecomputer . . . 31
A.2.2 Nood aan onderhoud van HLASM-code . . . 32
A.2.3 Moeilijkheden voor beginnende ontwikkelaars . . . 32
A.2.4 Effectiviteit van visualisatietechnieken . . . 32
A.2.5 Bestaande visualisatietools en parsers . . . 32
A.2.6 Control Flow in High Level Assembler . . . 33
A.2.7 JavaScript Parsing . . . 33
A.2.8 Mermaid.js en flowcharts . . . 34
A.3 Methodologie . . . 34
A.3.1 Fase 1: Oriëntatie en Literatuurstudie . . . 34
A.3.2 Fase 2: Requirementsanalyse . . . 35
A.3.3 Fase 3: Implementatie en Evaluatie . . . 35
A.3.4 Fase 4: Resultaat en Verdediging . . . 36
A.3.5 Tijdsplanning . . . 36
A.4 Verwacht resultaat, conclusie . . . 36

Bibliografie **37**

Lijst van figuren

2.1 De FermaT Maintenance Environment (FME). 7

4.1 De structuur van een statement in HLASM. 14
4.2 Machine Instructions 15
4.3 Assembler Instructions 15
4.4 Macro Instructions 16

Lijst van tabellen

5.1 Mapping van HLASM-elementen naar Mermaid.js 19

Lijst van codefragmenten

1

Inleiding

1.1. Probleemstelling

Vaak wordt een mainframe gezien als verouderde technologie die enkel gekend is uit films. Het tegengestelde is waar: mainframes zijn één van de meest geavanceerde servers die voortdurend onderhouden en gemoderniseerd worden. Mainframes zijn dan ook nog volop in gebruik. Door de betrouwbaarheid en capaciteit van een mainframe is het vooral geschikt voor grote aantallen transacties/aanvragen van gebruikers. Mainframes worden vaak gebruikt door banken, verzekeraars, luchtvaartmaatschappijen... om hun fundamentele bedrijfsprocessen te ondersteunen. Omdat deze systemen zo fundamenteel zijn, is het onderhoud ervan van groot belang. Vaak zijn de programma's die draaien op het mainframe ook tientallen jaren oud. Een deel van deze programma's is geprogrammeerd in High Level Assembler (HLASM). Deze programmeertaal wordt vaak gekozen omdat het programma snel moet zijn en directe controle over de hardware nodig heeft. Andere programma's maken vaak gebruik van de HLASM-programma's, waardoor ze ervan afhankelijk worden.

Daarbij zijn deze programma's vaak minimaal gedocumenteerd of bevatten ze verouderde documentatie. Hierdoor is het moeilijk om als nieuwe mainframe developer de voorgaande code te begrijpen.

Veel bedrijven hadden het idee dat mainframes onnodig zouden worden en dat ze konden vertrouwen op "modernere" servers. In werkelijkheid groeide hun workload alleen maar en werd die te groot om op reguliere servers te runnen. Met de voortdurende modernisatie bleek het mainframe echter de beste keuze om hun bedrijfsactiviteiten verder op te vertrouwen.

Met de oude mindset dat ze van het mainframe zouden weggaan, investeerden bedrijven niet in nieuw mainframe talent. Hierdoor bestaat de industrie vooral uit professionals die binnenkort met pensioen gaan. Nu het duidelijk is dat een main-

frame beter in staat is om hun huidige workload te beheren, is het tekort aan junior talent een probleem. De kennis van de bedrijfsprogramma's en systemen zit dus in een generatie die de arbeidsmarkt verlaat.
Hierdoor is er nood aan een nieuwe generatie om vooral het onderhoud van de systemen en programma's te beheren. Door het gebrek aan voldoende en bruikbare documentatie is het vaak moeilijk om als nieuwe mainframe programmeur code te begrijpen, wat het onderhouden ervan belemmert. Documentatie is dus belangrijk voor de leesbaarheid en onderhoudbaarheid van de code, waardoor minder fouten worden gemaakt.
Een hulpmiddel zou een visualisatietool kunnen zijn die kan dienen als documentatie. Beginners hebben vaak moeite met het begrijpen van de control flow van een programma, daarom is visualisatie van de control flow vooral handig.

1.2. De onderzoeksvraag

Dit onderzoek is gericht op het beantwoorden van de volgende vraag:

> *Hoe effectief versnellen statische analyse en Mermaid-diagrammen het leerproces van HLASM bij beginners?*

Binnen dit onderzoek wordt gefocust op twee aspecten: het technische en het menselijke. Er wordt gekeken naar hoe de visualisatie technisch kan geïmplementeerd worden en ook naar de effectiviteit van de visualisatie als documentatie en hulpmiddel bij het begrijpen van de control flow in HLASM.

1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is het uitwerken van een Proof-of-Concept en een bijhorende analyse ervan. De PoC moet in staat zijn de belangrijkste onderdelen van HLASM-code om te zetten in een Mermaid.js flowchart. De focus ligt expliciet op de structuur van het programma, dus minder op de details. Het moet bijvoorbeeld weergeven waar het programma begint, waar het zich opsplitst in verschillende takken en waar het eindigt.
Vooraleer de implementatie van start gaat, wordt onderzoek gedaan naar de bestaande technologieën die van hulp kunnen zijn. Dit onderzoek wordt uitgevoerd in de vorm van een literatuurstudie. Tijdens de literatuurstudie wordt gekeken naar het probleemdomein en het oplossingsdomein.
Binnen de literatuurstudie worden volgende vragen gesteld om het probleemdomein beter te verstaan:

- Waarom is een mainframe nog steeds van belang?
- Waarom is er nood aan het onderhouden van HLASM-code?
- Hoe helpt visualisatie met het begrijpen van codestructuur?

- Wat zijn de moeilijkheden bij het begrijpen van assemblytalen voor beginnende ontwikkelaars?

Rond het oplossingsdomein worden volgende vragen bestudeerd:

- Hoe helpt visualisatie bij het sneller begrijpen van programma's?
- Welke tools bestaan er al om HLASM-code te visualiseren?
- Welke HLASM parsers bestaan er?
- Welke JavaScript-libraries zijn toegankelijk en bruikbaar?

1.4. Het verwachte resultaat

Het verwachte resultaat van dit onderzoek is dat de visualisatie helpt bij het beheersen van HLASM-control flow, in tegenstelling tot het gebruik van manuele methodes. Om dit resultaat te meten wordt de PoC geëvalueerd aan de hand van testen die de tijd meten met de PoC en met de manuele manier.

1.5. Structuur van het onderzoek

Deze bachelorproef is als volgt opgesteld:
In Hoofdstuk 2 (Literatuurstudie) worden de deelvragen uit het oplossingsdomein en het probleemdomein onderzocht aan de hand van bestaande onderzoeken en literatuur. Hiermee wordt een beter zicht gecreëerd op het probleem en de oplossing om zo gericht mogelijk te werken.
In Hoofdstuk 3 (Methodologie) wordt toegelicht welke onderzoekstechnieken worden gebruikt.
In Hoofdstuk 4 (HLASM-analyse) wordt gekeken naar Assembler, hoe de programmeertaal is opgebouwd en welke elementen de structuur van het programma vastleggen.
In Hoofdstuk 5 (Control-flow visualisatie) wordt bekeken hoe de Assembler-structuur kan worden omgezet naar Mermaid.js-elementen. Er wordt onderzocht hoe dit visueel logisch kan worden gerepresenteerd.
In Hoofdstuk 6 (Proof-of-Concept) wordt de PoC uitgewerkt en daarna geëvalueerd aan de hand van testen.
In Hoofdstuk 7 (Conclusie) wordt gekeken naar het resultaat van de evaluatie en naar de literatuurstudie om een antwoord te formuleren op de onderzoeksvragen.

2

Literatuurstudie

2.1. De mainframeindustrie

Voor een lange tijd werd beweerd dat de mainframecomputer zou uitsterven en dat grote bedrijven er afstand van zouden doen (Sagers e.a., 2018). Het einde van mainframes is nog steeds niet gekomen en in sommige sectoren is er zelfs sprake van een trend om te blijven investeren in mainframes (Sagers e.a., 2018).
Het onderzoek van Sagers e.a. (2018) toont aan dat er een behoefte is aan een voortdurende werkracht met mainframe-kennis. Mainframes bekleden nog steeds een belangrijken rol in bedrijven en dat zal waarschijnlijk nog tientallen jaren zo blijven. Ook wordt vermeld dat de academische wereld niet in staat is om voldoende mensen met mainframe-kennis op te leiden. Dit komt omdat de opleiding in het verleden zijn stopgezet. Uit het onderzoek blijkt dat het aantal nieuwe medewerkers dat de industrie nodig heeft als gevolg van pensionering, groter is dan het aantal mensen dat wordt opgeleid door de academische wereld (Sagers e.a., 2018).
Het onderzoek van Sagers e.a. (2018) wordt ondersteund door recent onderzoek waarin wordt geconstateerd dat een mainframe voor grote IT-bedrijven nog steeds een belangrijke functie vervult. De mainframecomputer heeft daarbij ook nieuwe mogelijkheden gekregen die het aantrekkelijk maken voor nieuwe kopers. Het vermogen van mainframes om de fundamentele bedrijfsprocessen voor banken en verzekeringsmaatschappijen te waarborgen, samen met de nieuwe mogelijkheden die het de afgelopen jaren heeft gekregen, zorgen ervoor dat mainframes relevant blijven (Shaik, 2024).
Daarnaast wordt in het onderzoek van Waites en Ketterer (2013) het punt herhaald dat de werknemers die mainframes ontwikkelen, ondersteunen en onderhouden bijna met pensioen gaan en daarmee hun waardevolle kennis verdwijnt.

2.1.1. Modernisatie van legacy-systemen

Organisaties hebben verschillende mogelijkheden om legacy-systemen te moderniseren Sneed en Verhoef (2019). Ze hebben bijvoorbeeld de mogelijkheid tot conversie, herontwikkeling of herimplementatie van hun bestaande systemen. De studie concludeerd dat automatische conversie van bestaande systemen niet altijd succesvol is, vooral wanneer programma's sterk geïntegreerd zijn met de bestaande architecturen. Hierdoor kan herimplementatie een realistischer alternatief zijn.

2.1.2. Eigen Bijdrage

Deze studie is gericht op het verbeteren van programmacomprehensie van bestaande systemen. De focus ligt specifiek op het toegankelijk maken van bestaande HLASM-code door visualisate. Door de abstracte en vaak ongedocumenteerde aard van deze code, is het begrijpen van de logica een tijdrovend proces.

2.2. Aanwezigheid van HLASM-code

Veel software die vandaag op mainframes draait wordt beschouwd als legacysoftware. Programma's die vaak tientallen jaren geleden ontwikkeld zijn maar nog steeds een cruciale rol spelen binnen organisaties. Legacysoftware bevat meestal een grote hoeveelheid bedrijfslogica die moeilijk te reconstrueren is wanneer het systeem volledig zou worden vervangen (Sneed & Verhoef, 2019).
Door de hoge nauwkeurigheid, flexibiliteit en efficiëntie van HLASM, blijft het een essentieël component in mainframe-omgevingen (Yalamanchili, 2022). Het zorgt ervoor dat systeemontwikkelaars toegang hebben tot geheugen en het kunnen manipuleren. In hogere programmeertalen zoals COBOL, zijn deze functionaliteiten beperkt. HLASM wordt dus gebruikt binnen een context waar prestatie centraal staat, hierdoor blijft het een belangrijk component (Yalamanchili, 2022).

2.2.1. Onderhoud en Modernisatie

Applicaties moderniseren en onderhouden is een mogelijkheid die grote voordelen en lagere kosten kan bieden (Ehrman, 2004). Het legacy-systeem kan gezien worden als een investering die je hergebruikt. Ook heeft het minder nood aan nieuwe testen, sinds de code minder wordt veranderd integenstelling tot een volledige conversie (Ehrman, 2004).
Bij programmeertalen zoals assembler kan program comprehension complex zijn. De code bestaat uit elementaire instructies die direct met de hardware werken. Hierdoor is het moeilijker om de logica van het programma te identificeren zonder extra hulpmiddelen zoals analysetools of visualisaties (Baldwin e.a., 2009).
Tools kunnen helpen met het moderniseren van HLASM-applicaties (Ehrman, 2004). Ze kunnen namelijk meer inzicht geven in het gedrag van applicaties. Het moder-

niseren van HLASM kan stapsgewijs aangepakt worden: afzonderlijke instructies, blokken code, procedures, modules... De organisatie kiest zelf hoe groot of klein deze stappen zijn en per stap kunnen de nodige testen uitgevoerd worden (Ehrman, 2004).

2.2.2. Bestaande HLASM-hulpmiddelen

High Level Assembler Toolkit Feature

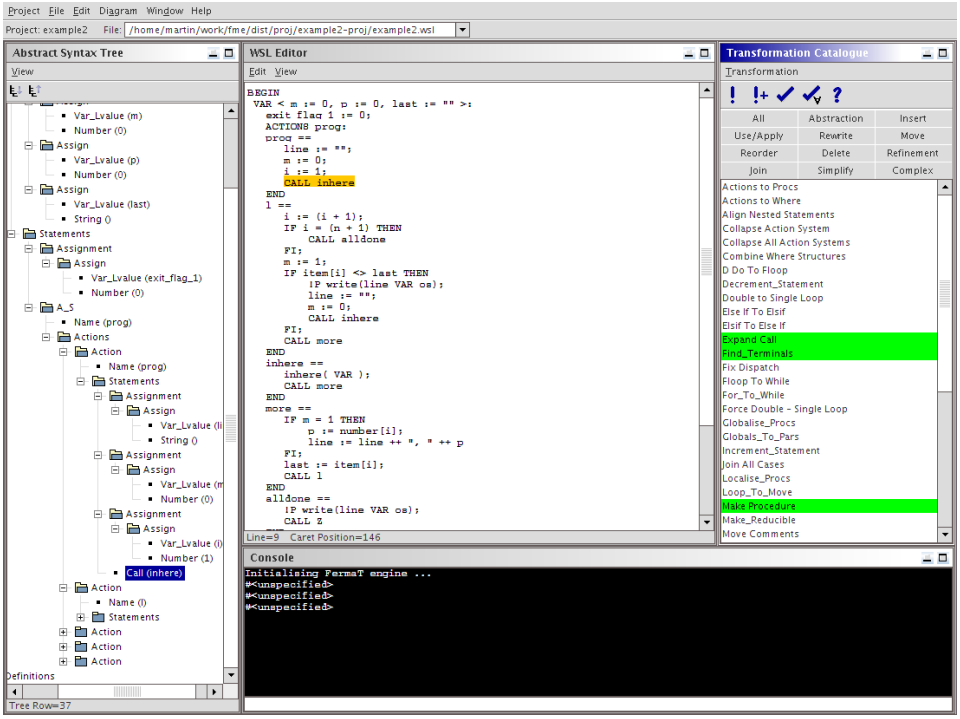
De High Level Assembler Toolkit Feature is een IBM-product dat een set van tools bevat om het herstel en de ontwikkeling van HLASM applicaties te verbeteren, analyseren, debuggen en te onderhouden (Ehrman, 2001). Hierdoor helpt het bij de modernisatie van HLASM-applicaties (Ehrman, 2004).
De tool is voornamelijk gericht op professionele mainframeontwikkelaars en vereist het gebruik van een volledige mainframeontwikkelomgeving. De proof-of-concept richt zich specifiek op het verbeteren van programmacomprehensie door middel van visuele representaties van HLASM-code.

FermaT

FermaT is een programma dat kan worden gebruikt voor de visualisatie van HLASM-code en dat ondersteuning biedt bij het analyseren en herstructureren van bestaande programma's. Het bestaat uit geïntegreerde tools voor programma-inzicht, migratie en herontwerp, waardoor ontwikkelaars beter zicht krijgen op complexe programma's (Ward, 2001).
In vergelijking met FermaT heeft de proof-of-concept een beperktere maar meer gerichte doelstelling, het visueel representeren van de control-flow van HLASM-code.

Tape-z

Tape/Z is een toolkit die werd ontworpen voor het analyseren en visualiseren van HLASM-code. Het doel van deze tool is om code overzichtelijk in kaart te brengen door middel van het parsen van instructies en het automatisch opbouwen van control-flow diagrammen die de structuur van het programma weergeven (avisheksen-gupta, 2025).
Tape/Z biedt veel nuttige functionaliteiten voor het analyseren van HLASM-code, maar het gebruik van de tool vereist een technische configuratie. Dit kan een drempel vormen voor ontwikkelaars of studenten die snel een visuele representatie willen genereren.
De proof-of-concept heeft als doel om een meer toegankelijke tool te zijn. De PoC wordt een webapplicatie die HLASM-code rechtstreeks omzet naar Mermaid.js syntax. Dit maakt het mogelijk om diagrammen te bouwen met tekstuele syntax die eenvoudig aangepast en uitgebreid kan worden. Hierdoor kunnen gebruikers de gegenereerde visualisaties niet alleen bekijken, maar ook verder aanpassen.



Figuur 2.1: Interface van de FermaT Maintenance Environment (FME). Links is de parse tree van het huidige programma zichtbaar, het middelste venster toont de WSL-code en rechts is de transformatiecatalogus (Ward, 2020).

Eigen Bijdrage

De focus van deze studie ligt op het ontwikkelen van een lichtgewicht en toegankelijke tool die specifiek gericht is op inzicht in control-flow. In tegenstelling tot bestaande oplossingen wordt gekozen voor een webgebaseerde aanpak.

2.2.3. Kosten van Onderhoud

In het onderzoek van Xia e.a. (2018) werd een grootschalig veldonderzoek gedaan naar hoe programmacomprehensie in de praktijk wordt uitgevoerd. Er werd gekeken naar 79 ontwikkelaars die aan 7 industriële projecten werkten. Als resultaat werd geconstateerd dat ontwikkelaars gemiddeld tot 58% van hun tijd besteden aan het begrijpen van programma's. Hierbij gebruiken ze vaak webbrowsers en documentatie editors. Ook werd vastgelegd dat senior ontwikkelaars minder tijd besteden aan het begrijpen van programma's en dat er meer tijd nodig voor is tijdens een onderhoudsfase (Xia e.a., 2018).

2.3. Moeilijkheden voor Beginnende Ontwikkelaars

Assemblytalen zijn zeer gedetailleerd, als ontwikkelaar definieer je elke kleine stap van het programma, wat ingewikkeld kan zijn voor beginners. Uit onderzoek blijkt dat naarmate de ervaring toeneemt, het gemakkelijker wordt om de functionaliteit van code te begrijpen (Feigenspan e.a., 2012).

Binnen het onderwijs hebben studenten vooral moeite met de componenten die de logische volgorde van het programma bepalen. Dit zijn de componenten die de control flow van het programma definiëren. Bij het begrijpen van de control flow maken beginners vaak fouten en ervaren ze vaak verwarring (Kawash, 2024).

2.3.1. Eigen Bijdrage

Visualisatie van control-flow kan deze moeilijkheden verminderen door de structuur van programma's zichtbaar te maken. Deze studie is specifiek om een tool te ontwikkelen die control-flow grafisch weergeeft vanuit HLASM-code.

2.4. Grafische Softwarevisualisatie

In het onderzoek van Cárdenas en Aponte (2017) werd een gecontroleerd experiment uitgevoerd om de impact van het gebruik van grafische visualisatietechnieken bij het begrijpen van software vast te leggen. Hiervoor werden tajen uitgevoerd door 20 studenten, de helft gebruike een visualisatietool, de andere helft niet. Er werd beordeeld op de voltooiingstijd en de nauwkeurigheid van de antwoorden. Uit het onderzoek werd geconcludeerd dat het gebruik van garfische visualisatie de juistheid verhoogt met 21,45%. Er was geen bewijs dat er een vermindering was van voltooiingstijd. Grafische visualisatietechnieken kunnen dus bijdragen aan het begrijpen van programma's (Cárdenas & Aponte, 2017).

2.4.1. Control-Flow Grafieken

Een control-flow grafiek heeft knooppunten die overeenkomen met basisblokken code met elk een gerichte rand die een mogelijke controlestroom tussen twee knooppunten aangeeft (Debbabi e.a., g.d.). Hierdoor is het mogelijk om alle uitvoeringspaden van een programma te analyseren. Ze zijn vooral heel nuttig voor statische analyse en om inzicht te verkrijgen in de structuur van een programma (Debbabi e.a., g.d.).

2.4.2. Mermaid.js

Mermaid.js is een open-source tool waarmee diagrammen en flowcharts worden opgesteld vanuit tekst. Het heeft een simpele syntax die het makkelijk maakt om visualisaties te maken. De tool wordt gebruikt voor documentatie, code-review en visuele uitleg (Sveidqvist & Jain, 2021).

2.4.3. Eigen Bijdrage

Binnen deze studie wordt Mermaid.js gebruikt als outputformaat. Hierdoor kunnen gegenereerde diagrammen eenvoudig aangepast worden en kan het geïntegreerd worden in documentatie.

2.5. Parsing van programmeertalen

Parsing is een fundamenteel onderdeel van de analyse en verwerking van programmeertalen. Tijdens het parsingproces wordt broncode geanalyseerd volgens een grammatica om de syntactische structuur van het programma te bepalen. Dit proces is een belangrijke stap binnen compilers, interpreters en analysetools (Zaytsev, 2020).

Een programmeertaal wordt meestal beschreven door grammaticaregels. Op basis van deze regels kan een parser bepalen of een stuk broncode syntactisch correct is en hoe de verschillende onderdelen van het programma met elkaar verbonden zijn. Het resultaat is vaak een boomstructuur, zoals een abstract syntax tree (AST), waarin de hiërarchische structuur van het programma wordt weergegeven (Sun e.a., 2023).

2.5.1. JavaScript Parsing

JavaScript is een geschikte programmeertaal om een parser mee te bouwen omdat het veel liberaries heeft en makkelijk integreert met webtechnologieën (Contributors, 2023). Daardoor kan de parsinglogica gebruikt worden voor analyse en directe visualisatie in een webinterface. JavaScript heeft ook een brede community en documentatie die het makkelijker maken om een parser te ontwikkelen (Contributors, 2023).

Chevrotain

Chevrotain is een krachtige JavaScript-parsing library die ontworpen is om efficiënte parsers te bouwen. In tegenstelling tot klassieke generatoren genereert Chevrotain geen externe code, maar laat je grammatica's volledig als gewone JavaScript-broncode definiëren (2018).

3

Methodologie

Binnen dit hoofdstuk wordt toegelicht hoe dit toegepast en technisch onderzoek werd uitgevoerd. Het onderzoeksproces werd uitgevoerd in verschillende fasen. Per fase wordt de doelstelling, de gebruikte onderzoeksmethode en het bijhorende resultaat toegelicht.

3.1. Fase 1: Literatuurstudie

Binnen de literatuurstudie werd onderzoek gedaan naar het probleemdomein en het oplossingsdomein. De doelstelling was om inzicht te krijgen in het belang van mainframes, de rol van HLASM en de moeilijkheden bij het begrijpen van assemblytalen voor beginnende ontwikkelaars. Daarnaast werd onderzocht welke visualisatietechnieken en bestaande tools gebruikt worden voor het analyseren van control flow.
De onderzoeksmethode in deze fase bestond uit het analyseren van wetenschappelijke literatuur, technische documentatie en bestaande tools.
Het resultaat van deze fase is Hoofdstuk 2 (Literatuurstudie), waarin de deelvragen worden beantwoord en de noodzaak van een visualisatietool wordt onderbouwd.

3.2. Fase 2: Technische analyse van HLASM

In deze fase werd de structuur van HLASM-broncode geanalyseerd. De doelstelling was het identificeren van structurele elementen die de control flow representeren, zoals labels, conditionele vertakkingen en sprongen.
De methode bestond uit een technische analyse van representatieve HLASM-codefragmenten. Hierbij werd specifiek gekeken naar de kolomgebaseerde structuur en naar patronen die gebruikt kunnen worden voor statische analyse.
Het resultaat van deze fase is Hoofdstuk 4 (HLASM-analyse), waarin wordt vastgelegd welke structurele elementen noodzakelijk zijn voor de visualisatie van de con-

trol flow.

3.3. Fase 3: Requirementsanalyse

Op basis van de literatuurstudie en de technische analyse werd een requirements-analyse opgesteld. De doelstelling was het definiëren van een afgebakende en technisch haalbare scope voor de proof-of-concept.
Volgende aspecten werden bepaald:

- Welke subset van HLASM-instructies representeert de control flow?
- Welke Mermaid.js-syntax vertaalt zich naar de juiste visualisatie van de control flow?
- Wat zijn de meetbare succescriteria voor de evaluatiefase?

Het resultaat van deze fase is Hoofdstuk 5 (Control-flow visualisatie), waarin een duidelijke afbakening en functionele eisen voor de verdere ontwikkeling en evaluatie staan beschreven.

3.4. Fase 4: Ontwerp en Implementatie

In deze fase werd de proof-of-concept ontwikkeld. De doelstelling was het automatisch omzetten van een geselecteerde subset HLASM-code naar een control-flowdiagram in Mermaid.js-syntax
De ontwikkeling gebeurde iteratief. Er werd een JavaScript-gebaseerde parser ontworpen die via patroonherkenning instructies en labels detecteert in de HLASM-broncode. Vervolgens werden deze gemapt naar Mermaid.js-elementen.
Het resultaat van deze fase is Hoofdstuk 6 (Proof-of-Concept), waarin het ontwerp en de implementatie wordt weergegeven.

3.5. Fase 5: Evaluatie

In de evaluatiefase werd onderzocht of de proof-of-concept effectief bijdraagt aan het sneller en correcter begrijpen van HLASM-control flow.
Er werd een vergelijkende studie uitgevoerd met 6 tot 10 beginnende mainframe-ontwikkelaars. De deelnemers analyseerden HLASM-codefragmenten, met of zonder gebruik van de PoC.
Om de correctheid te kunnen meten, werden volgende criteria vastgelegd:

- het identificeren van start- en eindpunten,
- het aanduiden van alle mogelijke vertakkingen,
- het uitleggen van een bepaalde conditionele vertakking.

Naast correctheid werden de resultaten ook geëvalueerd op de benodigde tijd om de code te analyseren.
Het resultaat van deze fase wordt besproken in Hoofdstuk 6 (Proof-of-Concept) en geïnterpreteerd in Hoofdstuk 7 (Conclusie).

4

HLASM-analyse

In dit onderzoek worden de termen assembler en HLASM als synoniemen gebruikt voor de IBM High Level Assembler (z/Architecture).

4.1. Structuur van een HLASM-programma

In dit hoofdstuk wordt de structuur van de assemblertaal beschreven en de componenten die het opmaken.

4.1.1. Een Assembler Statement

Een HLASM-programma bestaat uit een reeks assemblerstatements. Een statement is een regel in de broncode dat een instructie is of een regel commenteraar.

4.1.2. Instructies

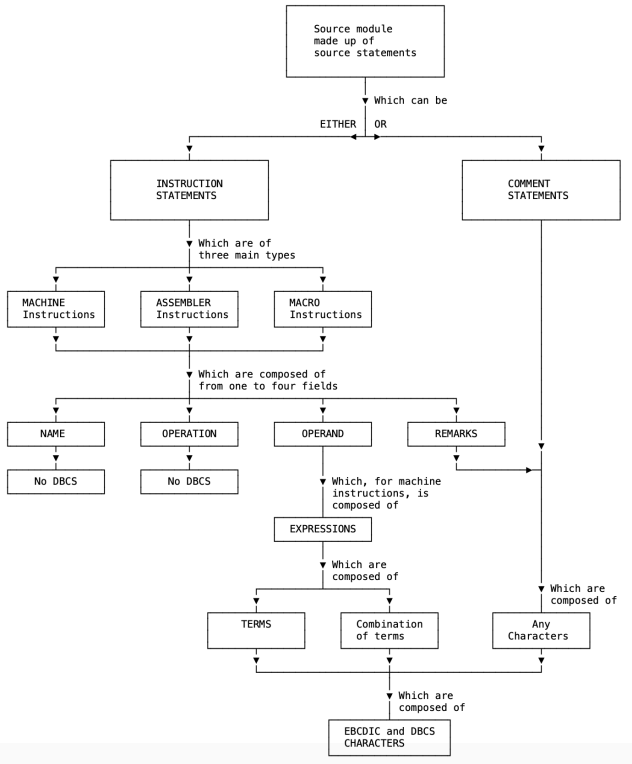
Er zijn drie verschillende types instructies:

Machine Instructions

Machine instructions zijn de feitelijke opdrachten voor de CPU. Ze worden door de assembler direct vertaald naar binaire objectcode die de processor kan uitvoeren. De syntaxis is gericht op het adresseren van registers en geheugenlocaties via expressies.

Veld	Status	Samenstelling (conform flowchart)
NAME	Optioneel	Een symbool of een spatie.
OPERATION	Verplicht	Een symbolische operatiecode (Mnemonic).
OPERAND	Varieert	Expressies, eventueel met base/index: `Exp`, `Exp(Exp)` of `Exp(Exp,Exp)`.

```
* Voorbeeld Machine Instructions
```



Figuur 4.1: De structuur van een statement in HLASM (2025)

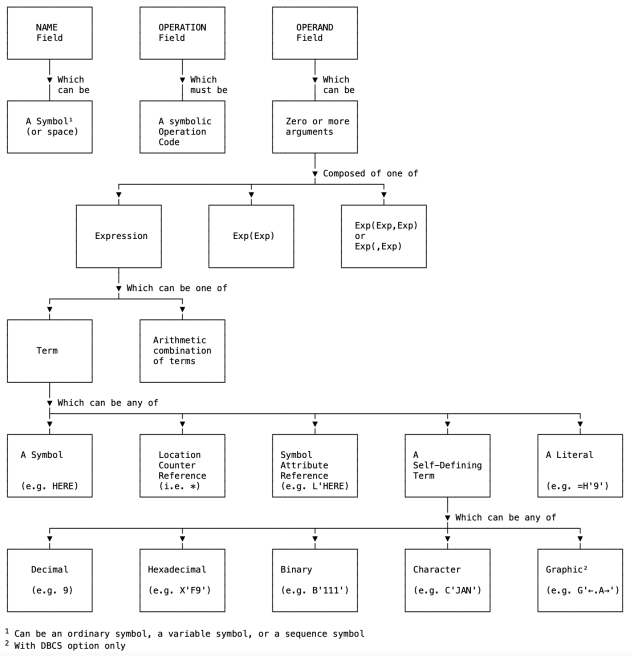
```
LABEL1   L      R1,DATA          Load: Exp (Symbool)
AR     R1,R2              Add Register
MVC    0(8,R4),=C'TEST' Exp(Exp,Exp) met Literal
```

Assembler Instructions

Assembler instructions (ook wel directives genoemd) zijn instructies aan de assembler zelf om de assemblage te sturen, zoals het definiëren van data of het reserveren van geheugen. Ze genereren niet altijd direct executable code.

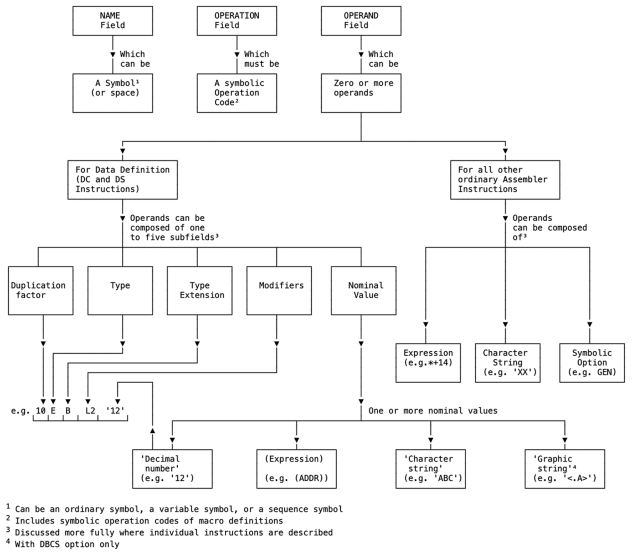
Veld	Status	Samenstelling (conform flowchart)
NAME	Optioneel	Een symbool of spatie.
OPERATION	Verplicht	Een symbolische operatiecode (Mnemonic).
OPERAND	Varieert	**Data Definitie:** Subvelden zoals Duplication Factor, Type, Modifiers en Nominal Value. **Overig:** Expressies, Character strings of Symbolic options.

```
 * Voorbeeld Assembler Instructions
DATA1   DC   3F'10'        Data Constant: Factor(3), Type(F), Value('10')
```



Figuur 4.2: Machine Instructions

```
BUFFER   DS    CL80            Data Storage: Type(C), Length(L80)
 TITLE 'Mijn Programma' Overige: Character String
```



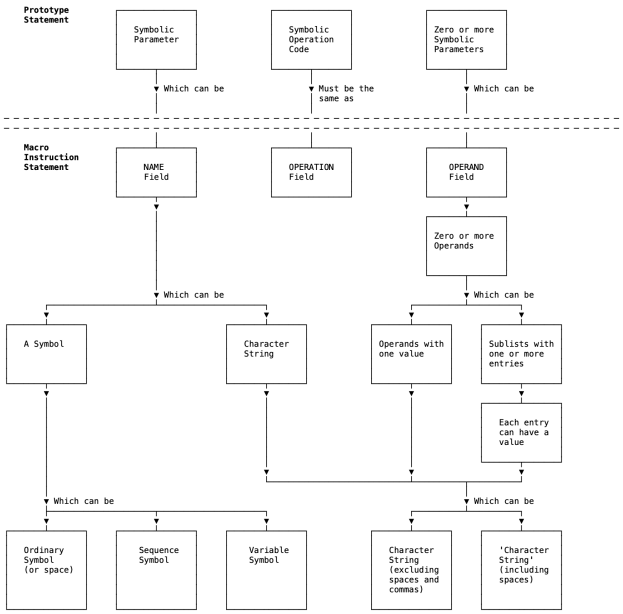
Figuur 4.3: Assembler Instructions

Macro Instructions

Macro instructions zijn verzoeken aan de assembler om een voorgedefinieerde set instructies (de macro-definitie) in de broncode te plaatsen. Deze worden tijdens de *pre-assembly* fase verwerkt.

Veld	Status	Samenstelling (conform flowchart)
NAME	Optioneel	Een symbool of een Character String (Ordinary/Sequence/Variable).
OPERATION	Verplicht	Moet exact overeenkomen met de Mnemonic in de Macro Prototype.
OPERAND	Optioneel	Operanden met één waarde of *sublists* met meerdere entries.

```
 * Voorbeeld Macro Instructions
 SAVE  (14,12)          Macro met een sublist als operand
MYMAC   OPEN  (INFILE,INPUT)  Macro met een Name field en sublist
 WTO   'Foutmelding'    Macro met een Character String
```



Figuur 4.4: Macro Instructions

4.2. Control-flow in HLASM

De control-flow van een programma beschrijft de volgorde waarin instructies worden uitgevoerd. In assemblerprogramma's is deze control-flow expliciet zichtbaar omdat spronginstructies direct verwijzen naar labels binnen de code. Hierdoor kan de uitvoering van een programma worden gemodelleerd als een reeks mogelijke uitvoeringspaden.

De meest eenvoudige vorm van control-flow is sequentiële uitvoering. Hierbij worden instructies één voor één uitgevoerd in de volgorde waarin ze in het programma voorkomen. De control-flow verandert wanneer een spronginstructie wordt uitgevoerd.

HLASM ondersteunt verschillende soorten spronginstructies. Onvoorwaardelijke sprongen, zoals B of BR, veranderen de programmateller naar het adres van een label of register zonder een voorwaarde te controleren. Conditionele sprongen, zoals

`BE` (Branch if Equal) of `BNE` (Branch if Not Equal), controleren eerst de waarde van de condition code voordat de sprong wordt uitgevoerd (*HLASM V1R6 Language Reference*, g.d.).
De control-flow van een assemblerprogramma kan dus beïnvloed worden door verschillende factoren, zoals spronginstructies, subroutineaanroepen en conditionele evaluaties. Deze elementen vormen samen de basis voor het analyseren van mogelijke uitvoeringspaden binnen een programma.

4.3. Selectie van control-flowrelevante instructies

5

Control-flow visualisatie

Dit hoofdstuk beschrijft hoe de control-flow van HLASM-programma's gevisualiseerd kan worden. Het doel van deze visualisatie is om ontwikkelaars een overzicht te geven bij de mogelijke uitvoeringspaden binnen een HLASM-programma. Een grafische visualistie maakt het eenvoudiger om sprongen, vertakkingen en programmastructuren te analyseren.

5.1. Requirements

Voor het ontwikkelen van een proof-of-concept moeten de functionele en niet-functionele vereisten worden gedefinieerd. Deze vereisten bepalen welke onderdelen van HLASM-code worden ondersteund en hoe de visualisatie eruit moet zien.

5.1.1. Functionele vereisten

Voor de PoC zijn de volgende functionele vereisten opgesteld:

- Detectie van labels in de broncode, zodat mogelijke sprongdoelen geïdentificeerd kunnen worden.
- Detectie van spronginstructies die de control-flow van het programma wijzigen, zoals onvoorwaardelijke en conditionele branches.
- Correcte mapping van labels naar nodes in de control-flowgrafiek.
- Correcte mapping van sprongen naar edges tussen nodes.
- Ondersteuning voor zowel sequentiële uitvoering als vertakkingen binnen het programma.

5.1.2. Niet-functionele vereisten

Voor de PoC zijn de volgende niet-functionele vereisten opgesteld:

- De gegenereerde diagrammen moeten leesbaar zijn en een duidelijk overzicht geven van de programmastructuur.
- De visualisatie moet automatisch gegenereerd worden.
- De output moet weergegeven worden in de syntaxis van Mermaid.js, zodat de diagrammen eenvoudig aangepast kunnen worden.
- De gegenereerde Mermaid.js-code moet correct en syntactisch geldig zijn.

5.2. Ontwerp

Het ontwerp van de visualisatietool bestaat uit een vertaling van HLASM-broncode naar een grafische representatie van de control-flow.

5.2.1. Mapping van HLASM naar Mermaid.js

HLASM-element	Mermaid.js representatie
Label	Node in de flowchart
Sequentiële instructie	Verbinding tussen twee nodes
Onvoorwaardelijke sprong	Edge naar doelnode
Conditionele sprong	Vertakkende edge met conditie
Programma start	Entry node

Tabel 5.1: Mapping van HLASM-elementen naar Mermaid.js

6

Proof-of-Concept

6.1. Architectuur van de applicatie

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.

Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

6.2. Beperkingen van de implementatie

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.
Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.
Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

6.3. Evaluatie

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.
Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.
Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis

magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.

Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

6.3.1. Titel A

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mau-

ris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.
Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.
Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.
Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.
Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

6.3.2. Titel B

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede,

suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.
Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.
Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.
Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.
Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.
Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

6.3.3. Titel C

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.

Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum

ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

7

Conclusie

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Quisque enim. Proin velit neque, tristique eu, eleifend eget, vestibulum nec, lacus. Vivamus odio. Duis odio urna, vehicula in, elementum aliquam, aliquet laoreet, tellus. Sed velit. Sed vel mi ac elit aliquet interdum. Etiam sapien neque, convallis et, aliquet vel, auctor non, arcu. Aliquam suscipit aliquam lectus. Proin tincidunt magna sed wisi. Integer blandit lacus ut lorem. Sed luctus justo sed enim.

Morbi malesuada hendrerit dui. Nunc mauris leo, dapibus sit amet, vestibulum et, commodo id, est. Pellentesque purus. Pellentesque tristique, nunc ac pulvinar adipiscing, justo eros consequat lectus, sit amet posuere lectus neque vel augue. Cras consectetuer libero ac eros. Ut eget massa. Fusce sit amet enim eleifend sem dictum auctor. In eget risus luctus wisi convallis pulvinar. Vivamus sapien risus, tempor in, viverra in, aliquet pellentesque, eros. Aliquam euismod libero a sem.
Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.

A

Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Tegenwoordig spelen mainframecomputers een cruciale rol in de dagelijkse activiteiten van bedrijven in de financiële sector, de gezondheidszorg, verzekeringen en tal van andere publieke en private ondernemingen. Deze bedrijven zijn afhankelijk van mainframecomputers en hebben dan ook de nood om deze zo optimaal mogelijk te onderhouden. Vermits veel ervaren mainframe-specialisten hun pensioen naderen, is er nood aan een nieuwe generatie mainframe-specialisten die het onderhoud van hun systemen kan waarborgen. Onder dit onderhoud vallen de bedrijfskritische functionaliteiten van mainframeapplicaties, die voornamelijk afhankelijk zijn van High-Level Assembler (HLASM). Deze legacycode is vaak tientallen jaren oud en minimaal gedocumenteerd. Voor de nieuwe generatie mainframeontwikkelaars vormt dit een uitdaging om deze legacycode te begrijpen en te onderhouden. Zij vertrouwen vaak enkel op manuele code reviews.
Door middel van een geautomatiseerde tool die statische analyse van HLASM-code visualiseert in een control-flowchart, wil dit onderzoek ondersteuning bieden aan beginnende mainframeontwikkelaars, specifiek binnen organisaties die afhankelijk zijn van legacy HLASM-code.
De probleemstelling van dit onderzoek luidt als volgt:

> *Hoe effectief versnellen statische analyse en Mermaid-diagrammen het leerproces van HLASM bij beginners?*

Het doel van dit onderzoek is het ontwikkelen van een proof-of-concept (PoC) die

een representatieve subset van HLASM-instructies en labels statisch analyseert en deze omzet in een visueel control-flowdiagram in Mermaid.js-syntax.
Vooraleer de ontwikkeling van de PoC van start gaat, wordt een literatuurstudie uitgevoerd naar het probleemdomein en het oplossingsdomein.
Rond het probleemdomein worden volgende vragen bestudeerd:

- Wat is het belang van de mainframecomputer?
- Waarom is er nood aan het onderhouden van HLASM-code?
- Wat zijn de moeilijkheden bij het begrijpen van assemblytalen voor beginnende ontwikkelaars?

Rond het oplossings domein worden volgende vragen bestudeerd:

- Hoe helpt visualisatie bij het sneller begrijpen van programma's?
- Welke tools bestaan er al om HLASM-code te visualiseren?
- Welke HLASM parsers bestaan er?
- Welke HLASM-instructies zijn van noodzaak om de control-flow van de code in kaart te brengen?
- Wat is de beste manier om de kolom-gebaseerde structuur van HLASM te verwerken met een JavaScript-parser?
- Hoe vertaal je de vertakkingen in de HLASM-code naar correcte Mermaid.js-diagrammen?

A.2. Literatuurstudie

A.2.1. Belang van de mainframecomputer

Vaak worden mainframes beschreven als oude technologie en wordt gedacht dat ze niet meer relevant zijn. Ook wouden bedrijven graag mainframes vervangen door andere servers. Momenteel zijn mainframes nog steeds zeer actief en staan ze in voor kritische bedrijfsactiviteiten. Tegenwoordig worden mainframes ook geïntegreerd met moderne technologieën (Sagers e.a., 2018). Het onderzoek van Sagers e.a. (2018) wordt bevestigd door recent onderzoek waarin wordt aangetoond dat organisaties niet enkel gebruikmaken van legacyapplicaties, maar ook bezig zijn met het actief ontwikkelen van nieuwe applicaties (Shaik, 2024).
Mainframes zijn dus een zeer relevante en moderne technologie, die dagelijks draait. Omdat ervaren mainframeprofessionals hun pensioen naderen is er een behoefte aan een nieuwe generatie mainframespecialisten die het onderhoud van deze systemen kunnen waarborgen (Waites & Ketterer, 2013).

A.2.2. Nood aan onderhoud van HLASM-code

Bedrijfskritische functionaliteiten van mainframeapplicaties zijn vaak afhankelijk van High Level Assembler (HLASM), waardoor onderhoud van deze legacycode van groot belang is. Als de aangeroepen HLASM-code faalt, dan faalt het volledige programma (Zaytsev, 2020). Uit een onderzoek naar softwareonderhoud blijkt dat het begrijpen van bestaande code een groot deel van de moeite neemt tijdens onderhoud. Vooral bij beginnende ontwikkelaars gaat het merendeels van hun tijd aan het begrijpen van de functionaliteit, dit is omdat zij minder ervaring hebben (Xia e.a., 2018). Dit is vooral het geval bij legacycode zoals HLASM, omdat daar de documentatie vaak beperkt is.

A.2.3. Moeilijkheden voor beginnende ontwikkelaars

Assemblytalen zijn zeer gedetailleerd, als ontwikkelaar definieer je elke kleine stap van het programma, wat ingewikkeld kan zijn voor beginners. Uit onderzoek blijkt dat naarmate de ervaring toeneemt, het gemakkelijker wordt om de functionaliteit van code te begrijpen (Feigenspan e.a., 2012).
Binnen het onderwijs hebben studenten vooral moeite met de componenten die de logische volgorde van het programma bepalen. Dit zijn de componenten die de control flow van het programma definiëren. Bij het begrijpen van de control flow maken beginners vaak fouten en ervaren ze vaak verwarring (Kawash, 2024). Hieruit kan worden geconcludeerd dat de control flow van code een moeilijkheid is onder beginnende ontwikkelaars.

A.2.4. Effectiviteit van visualisatietechnieken

Visualisatietechnieken worden vaak gebruikt om code beter te begrijpen en worden ook erkend als technieken die sneller en meer inzicht geven in codefunctionaliteiten. Visualisaties worden vaak gebruikt om de structuur van code te representeren (Storey, 2005). Bij assemblytalen kan visualisatie vooral helpen, aangezien de code vaak is opgebouwd uit zeer gedetailleerde instructies en kan worden gegroepeerd in componenten (Baldwin e.a., 2009).

A.2.5. Bestaande visualisatietools en parsers

Een bestaande tool die kan worden gebruikt voor de visualisatie van HLASM-code is FermaT. Deze tool is vooral gericht op het begrijpen en migreren van legacycode. Het is een zeer krachtige en zware tool die veel functionaliteiten heeft (Ward, 2001). Ook bied IBM een HLASM parser aan, die onderdeel maakt van IBM Developer for z/OS. Deze parser is bedoeld voor diepe analyse en gebruik met andere tools (IBM, g.d.).
Het doel van dit onderzoek is het ontwikkelen van een lichte tool die HLASM-code omzet naar een high-level control-flowdiagram visualisatie in Mermaid.js-syntax. Deze tool is specifiek bedoeld voor educatieve doeleinden, om beginnende ont-

wikkelaars te ondersteunen. Ook ligt de focus enkel op een beperkte subset van control-flow-instructies, in tegenstelling tot bestaande tools.

A.2.6. Control Flow in High Level Assembler

High Level Assembler (HLASM) wordt gezien als een Second Generation Language (2GL), de instructies in HLASM komen rechtstreeks overeen met processor-commando's. Er zijn dus geen abstracties zoals functies of objecten. Deze abstracties zijn wel terug te vinden in talen met een hoger generatieniveau zoals COBOL. HLASM biedt daardoor veel controle over het fysieke systeem, maar dat maakt de code vaak moeilijker te lezen en te onderhouden (Kornelis, 2003).
Om de control-flow van HLASM te definiëren kan er gefocust worden op bepaalde elementen die sprongen of vertakkingen maken:

- Een directe sprong: Instructies zoals B (Branch) sturen het programma direct naar een ander codeblok (*High Level Assembler for z/OS & z/VM & z/VSE: HLASM V1R6 Programmer's Guide*, g.d.).
- Een conditionele sprong: Door middel van vergelijkingen (C voor Compare) of condities (BE voor Branch on Equal) verandert de route van het programma (*High Level Assembler for z/OS & z/VM & z/VSE: HLASM V1R6 Programmer's Guide*, g.d.).
- Een subroutine: Bij het aanroepen van een subroutine worden instructies zoals BAL of BAS gebruikt. Het terugkeren vanuit een subroutine gebeurt via een BR-instructie (Ward, 2013).

Deze specifieke instructies kunnen een idee geven van de control-flow van het programma.

A.2.7. JavaScript Parsing

Binnen HLASM wordt er gewerkt met een vaste kolom-gebaseerde indeling. Elk onderdeel van de code staat op een vaste plek. Elke kolom heeft een eigen functie: kolommen 1-8 zijn voor labels, kolommen 10-14 voor de instructie en vanaf kolom 16 staan de operanden. Na kolom 71 wordt alles beschouwd als commentaar of wordt het genegeerd (*HLASM V1R6 Language Reference*, g.d.).
JavaScript is zeer nuttig om tekst te parsen, het is namelijk erg goed in het doorzoeken en herkennen van patronen (Flanagan, 2011). Voor de PoC is een regel-voor-regel aanpak voldoende. Hierbij zal de parser elke regel doorlopen op basis van de vaste kolomposities. Zo kunnen de labels en instructies herkend worden die bijdragen aan de control-flow. De verzamelde elementen worden vervolgens gemapt naar Mermaid.js-elementen.

A.2.8. Mermaid.js en flowcharts

Met Mermaid.js worden flowcharts en andere structuren vanuit tekst automatisch gevisualiseerd (Sveidqvist & Jain, 2021). Dit maakt het makkelijk om een mapping te maken van HLASM-elementen naar een visuele representatie. Het gebruik van flowcharts om complexe software begrijpelijk te maken is een bekend middel in de informatica (Ensmenger, 2016).
Een flowchart maakt gebruik van blokken en pijlen om een route te definiëren. In de context van informatica worden flowcharts vaak gebruikt om de route van een programma weer te geven (Charntaweekhun & Wangsiripitak, 2006). In het geval van HLASM kunnen bijvoorbeeld sprongingstructies worden weergegeven als pijlen die het programma brengen naar een ander codeblok.

A.3. Methodologie

Deze bachelorproef is een toegepast en technisch onderzoek waarin een oplossing wordt gezocht voor het onderhouden en begrijpen van legacy HLASM-broncode. Het onderzoek start met een literatuurstudie naar het probleemdomein en het oplossingsdomein. Ook wordt de control flow van HLASM-broncode bestudeerd, waarna een subset van representatieve HLASM-instructies en labels geselecteerd worden. Na de literatuurstudie start de ontwikkeling van een technische proof-of-concept (PoC) die de geselecteerde subset visueel omzet in een control-flowchart in Mermaid.js-syntax.

A.3.1. Fase 1: Oriëntatie en Literatuurstudie

Tijdens deze fase wordt gefocust op het aanscherpen van de probleemstelling en het oplossingsdomein. Dit gebeurt door middel van een literatuurstudie waarin dieper wordt ingegaan op de deelvragen:

- Wat is het belang van de mainframecomputer?
- Waarom is er nood aan het onderhouden van HLASM-code?
- Wat zijn de moeilijkheden bij het begrijpen van assemblytalen voor beginnende ontwikkelaars?
- Wat is de effectiviteit van visualisatietechnieken voor het sneller begrijpen van programma's?
- Wat zijn de bestaande visualisatietools binnen High Level Assembler?
- Welke HLASM-instructies zijn van noodzaak om de control-flow van de code in kaart te brengen?
- Wat is de beste manier om de kolom-gebaseerde structuur van HLASM te verwerken met een JavaScript-parser?

- Hoe vertaal je de vertakkingen in de HLASM-code naar correcte Mermaid.js-diagrammen?

Deze fase loopt van 20 december 2025 tot 1 maart 2026 en resulteert in een uitgewerkte literatuurstudie en methodologie. Het resultaat bevat concreet een theoretisch onderzoek naar de probleemstelling en oplossingsdomein, en een afgebakende scope van HLASM-instructies en labels voor de verdere ontwikkeling van de technische oplossing.

A.3.2. Fase 2: Requirementsanalyse

Tijdens de literatuurstudie (tussen januari 2026 en maart 2026) wordt een requirementsanalyse uitgevoerd. Het doel van deze fase is om concrete en meetbare eisen voor de proof of concept en de evaluatie vast te leggen.
Binnen deze fase beginnen we eerst met het specifiek definiëren van de doelgroep. Uit de doelgroep wordt gezocht naar 6 à 10 deelnemers die instaan voor de evaluatie van de PoC. Voor de evaluatie worden ook HLASM-codefragmenten verzameld die typische control-flowconstructies hebben zoals labels, conditionele takken en subroutines-aanroepen.
Ook worden de succescriteria van de PoC vastgesteld:

- correcte detectie van de geselecteerde HLASM-instructies en labels;
- correcte omzetting naar Mermaid.js-flowcharts;
- meetbare tijdswinst bij het begrijpen van de control flow in vergelijking met een manuele analyse.

A.3.3. Fase 3: Implementatie en Evaluatie

Tijdens de implementatie- en evaluatiefase wordt de proof-of-concept uitgewerkt en iteratief geëvalueerd. Deze fase vindt plaats tussen 2 maart 2026 en 4 mei 2026. De ontwikkeling gebeurt volgens een scrum-werkwijze om de effectiviteit van de PoC te waarborgen. Met behulp van de subset die in de vorige fase werd gedefinieerd, wordt een JavaScript-gebaseerde parser ontwikkeld die HLASM-code statisch analyseert. De parser identificeert instructies en labels aan de hand van patroonherkenning in de structuur van HLASM-broncode en zet deze via een mapping om naar Mermaid.js-syntax.
Concreet worden tijdens deze fase de volgende aspecten ontwikkeld:

- de logische opbouw van de parser op basis van patroonherkenning in de HLASM-broncodestructuur;
- een mapping van de geselecteerde HLASM-instructies en labels naar Mermaid.js-elementen;

- een evaluatie van de PoC op correctheid en tijdswinst.

Tijdens de ontwikkeling wordt de PoC iteratief geëvalueerd via een vergelijkende studie met handmatige code reviews. De evaluatie wordt uitgevoerd door 6 à 10 vrijwillige deelnemers, de helft van de deelnemers gebruikt de PoC om codefragmenten uit te leggen en de andere helft doet dit zonder de PoC. Beide groepen maken gebruik van dezelfde codefragmenten en worden getimed op het moment waarop zij zelf aangeven dat ze de code begrijpen.
De evaluatie loopt tijdens de implementatie, van 1 april 2026 tot 4 mei 2026.

A.3.4. Fase 4: Resultaat en Verdediging

De laatste fase loopt van 4 mei 2026 tot juni 2026. In deze fase worden de onderzoeksresultaten en evaluatieresultaten verwerkt tot een conclusie.
Specifiek worden de volgende aspecten besproken:

- de sterktes en beperkingen van de PoC;
- mogelijke uitbreidingen voor toekomstig onderzoek.

Deze fase resulteert in de finale bachelorproef die wordt ingediend op 29 mei 2026, de verdediging en de proof-of-concept met de bijbehorende documentatie.

A.3.5. Tijdsplanning

Elke fase heeft een tijdsperiode waarin de specifieke resultaten moeten worden opgeleverd voor de volgende fase. Zoals te zien in figuur **??**, zijn ook de administratieve deliverables opgenomen.

A.4. Verwacht resultaat, conclusie

Het verwachte resultaat van deze studie is een proof of concept (PoC), die in staat is een subset van representatieve HLASM- en labels om te zetten in visuele stappen geïnterpreteerd in Mermaid.js-syntax. De PoC brengt door middel van statische HLASM-broncodeanalyse de control flow in kaart. Met deze PoC wordt verwacht beginnende mainframeontwikkelaars te ondersteunen met het begrijpen van een onbekend stuk HLASM-broncode. De evaluatie van het verwachte resultaat wordt uitgevoerd door middel van een vergelijking met de traditionele, handmatige manieren van code reviews.

Bibliografie

(2018). https://chevrotain.io/docs/

(2025, oktober). https://www.ibm.com/docs/en/hla-and-tf/1.6.0?topic=structure-overview-assembler-language

avishek-sen-gupta. (2025). Tape/Z.

Baldwin, J., Myers, D., Storey, M.-A., & Coady, Y. (2009). Assembly Visualization and Analysis: An Old Dog CAN Learn New Tricks! https://ecs.wgtn.ac.nz/foswiki/pub/Events/PLATEAU/2009Program/plateau09-baldwin.pdf

Cárdenas, G. O., & Aponte, J. (2017). Evaluating the Graph-based Visualization Technique: A Controlled Experiment. *Enfoque UTE*, *8*(1), 201–216. https://www.redalyc.org/journal/5722/572262176015/html/

Charntaweekhun, K., & Wangsiripitak, S. (2006). Visual Programming using Flowchart. *2006 International Symposium on Communications and Information Technologies*, 1062–1065. https://doi.org/10.1109/ISCIT.2006.339940

Contributors, M. (2023, september). JavaScript. https://developer.mozilla.org/en-US/docs/Web/javascript

Debbabi, M., Salcic, Z., Biglari-Abhari, M., & Malik, A. (g.d.). Control-Flow Graph - an overview | ScienceDirect Topics. https://www.sciencedirect.com/topics/computer-science/control-flow-graph

Ehrman, J. (2001). *High Level Assembler: Toolkit Feature Technical Overview Synopsis*. https://public.dhe.ibm.com/software/websphere/awdtools/hlasm/s8166a.pdf

Ehrman, J. (2004). *Extending the Life Cycle of Legacy Applications*. https://public.dhe.ibm.com/software/websphere/awdtools/hlasm/S8132JEA.pdf

Ensmenger, N. (2016). The Multiple Meanings of a Flowchart. *51*(3), 321–351. Verkregen januari 25, 2026, van http://www.jstor.org/stable/44667617

Feigenspan, J., Kästner, C., Liebig, J., Apel, S., & Hanenberg, S. (2012). Measuring programming experience. *2012 20th IEEE International Conference on Program Comprehension (ICPC)*, 73–82. https://doi.org/10.1109/ICPC.2012.6240511

Flanagan, D. (2011). *JavaScript: The Definitive Guide* (6th). O'Reilly Media.

High Level Assembler for z/OS & z/VM & z/VSE: HLASM V1R6 Programmer's Guide [SC26-4941-07]. (g.d.). IBM Corporation.

HLASM V1R6 Language Reference [SC26-4940-06]. (g.d.). IBM Corporation.

IBM. (g.d.). *HLASM Parser* [IBM Developer for z/OS documentation]. Verkregen januari 27, 2026, van https://www.ibm.com/docs/en/developer-for-zos/16.0.x?topic=files-hlasm-parser

Kawash, J. (2024). Where Do Students Struggle Most in a First Course on Assembly Language? *Proceedings of the 26th Western Canadian Conference on Computing Education*. https://doi.org/10.1145/3660650.3660652

Kornelis, A. F. (2003). HLASM - Why assembler? https://bixoft.nl/english/why.htm

Sagers, G., Ball, K., Hosack, B., Twitchell, D., & Wallace, D. (2018). The Mainframe Is Dead. Long Live the Mainframe! *AIS Transactions on Enterprise Systems*, *2*(1). https://www.aes-journal.com/index.php/ais-tes/article/view/6

Shaik, S. (2024). Importance of Mainframe in Modern Era of Technologies. *Journal of Engineering and Applied Sciences Technology*, 1–3. https://doi.org/https://doi.org/10.47363/jeast/2024(6)257

Sneed, H., & Verhoef, C. (2019). Re-implementing a legacy system. *Journal of Systems and Software*, *155*. https://doi.org/https://doi.org/10.1016/j.jss.2019.05.012

Storey, M.-A. (2005). Theories, methods and tools in program comprehension. *IEEE Software*, *22*(3), 36–45.

Sun, W., Fang, C., Miao, Y., You, Y., Yuan, M., Chen, Y., Zhang, Q., Guo, A., Chen, X., Liu, Y., & Chen, Z. (2023, december). Abstract Syntax Tree for Programming Language Understanding and Representation: How Far Are We? https://doi.org/https://doi.org/10.48550/arXiv.2312.00413

Sveidqvist, K., & Jain, A. (2021). *The official guide to Mermaid.js: Create complex diagrams and beautiful flowcharts easily using text and code*. Packt Publishing.

Waites, S., & Ketterer, J. (2013). Lack of Mainframe Programmers a Critical Issue for IT Organizations A Review of the Literature. *International Journal of Business, Humanities and Technology*, *3*(7). https://ijbht.thebrpi.org/journals/Vol_3_No_7_September_2013/3.pdf

Ward, M. (2013). Assembler restructuring in FermaT. *2013 IEEE 13th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 147–156. https://doi.org/10.1109/SCAM.2013.6648196

Ward, M. (2020). FermaT. https://www.gkc.org.uk/fermat.html

Ward, M. (2001). The FermaT assembler re-engineering workbench. *Proceedings IEEE International Conference on Software Maintenance. ICSM 2001*, 659–662. https://doi.org/10.1109/ICSM.2001.972783

Xia, X., Bao, L., Lo, D., Xing, Z., Hassan, A. E., & Li, S. (2018). Measuring Program Comprehension: A Large-Scale Field Study with Professionals. *IEEE Transactions on Software Engineering*, *44*(10), 951–976. https://doi.org/10.1109/TSE.2017.2734091

Yalamanchili, C. M. (2022). Technical Insights into IBM High-Level Assembler (HLASM) in z/OS Systems. *International Journal of Multidisciplinary Research and Growth Evaluation*, *3*(1), 1155–1162. https://doi.org/https://doi.org/10.54660/.ijmrge.2022.3.1.1155-1162

Zaytsev, V. (2020). Journal of Object Technology Published by AITO -Association Internationale pour les Technologies Objets Vadim Zaytsev. Modelling of Language Syntax and Semantics: The Case of the Assembler Compiler. *Journal of Object Technology*, *19*(1), 1–22. https://grammarware.net/text/2020/hlasm.pdf